

TwinCAT 4026, Programming Conveyor 01 and Stop 01.

In this chapter we will program the code to automate the Conveyor 01 and Stop 01 from the Model. We have different sensors and actuators on this section.

List of sensors:

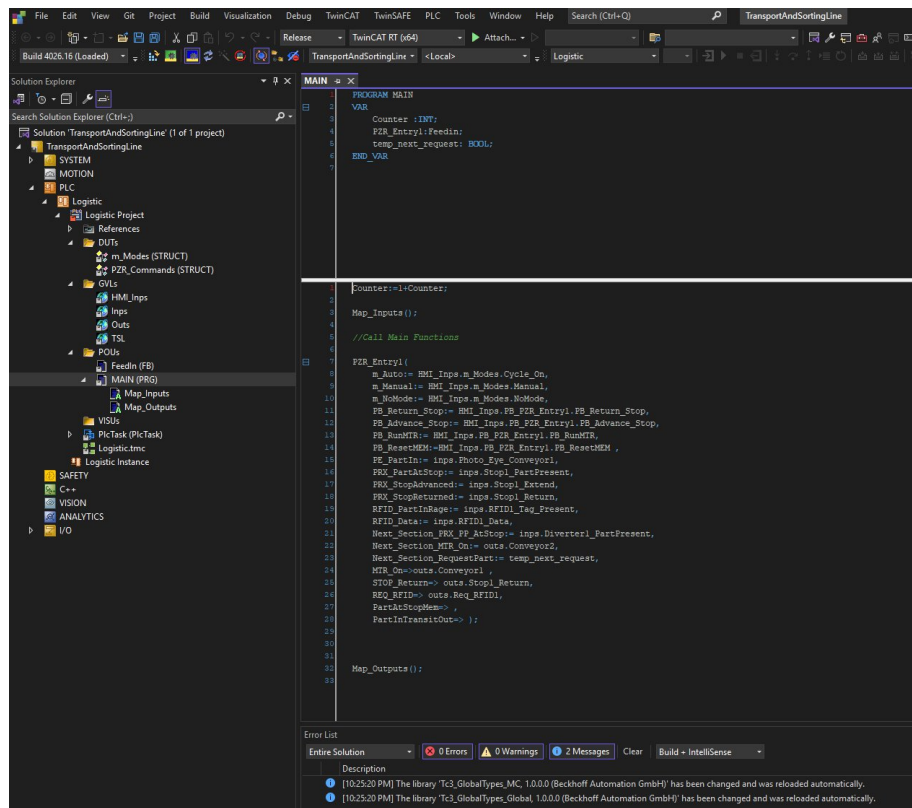
- Diverter 01 Part Present
- Diverter 01 Part In Transit
- Storage Blue Part Present
- Storage White Part Present

List of actuators:

- Diverter 01 Conveyor Motor
- Diverter 01 Up Motor
- Diverter 01 Down Motor.

Retrieve Chapter 03 Project.

We need to retrieve the file from Chapter 03. This file already has the first conveyor and stop logic.



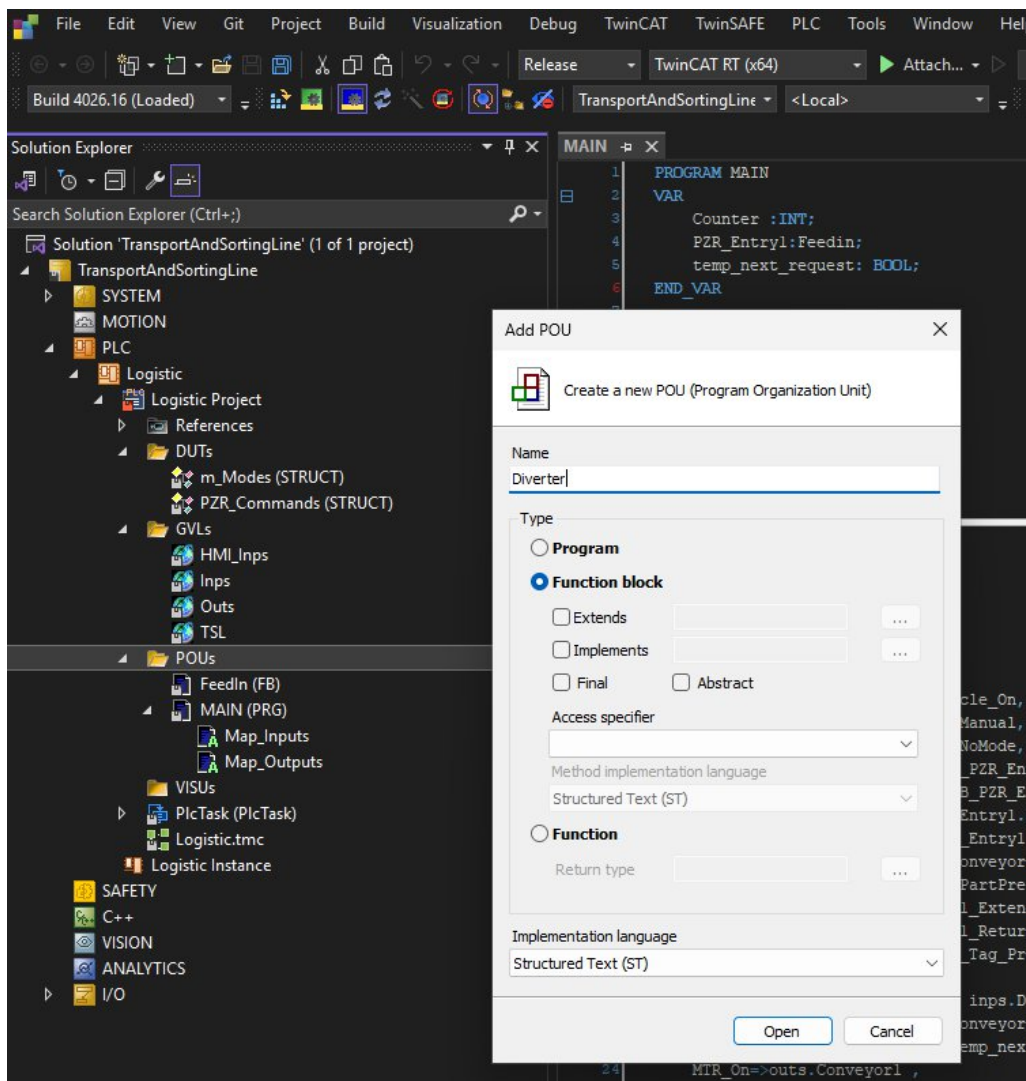
Diverter Function Block

We need to automate the Diverter 01. The best way to do it is creating our own function block. This function block will have the same behavior as TwinCAT standard libraries. It will have inputs, outputs and local variables. We will use the same basic instructions from the previous chapter.

The next step is to create a new Function Block.

Right click on the POU folder, select add and click POU.

Select Function Block and name it Diverter



The Diverter Function Block require multiple input signals.

For example the sensors that Diverter 01 has assigned. HMI Push Buttons to perform certain task in manual. Operational Mode of the equipment (Auto, Manual. Cycle On). We can see how separating the signals of the function block in inputs, outputs and local variables is very useful to organize the program.

Diverter Input variables.

VAR_INPUT

//Operation Mode

m_Auto:BOOL; *//m_Cycle - Auto and Cycle Running*
m_Manual:BOOL; *//m_Manual - Manual Operation Only*
m_NoMode:BOOL; *//m_NoMode - No Machine mode selected*

//Manual Functions

PB_UpMtr:BOOL; *//Push Button to Return Stop*
PB_DownMtr:BOOL; *//Push Button to Advance Stop*
PB_RunMTR:BOOL; *//Push button to Run Motor*
PB_ResetMEM:BOOL; *//Push Button to Reset Memory of Part Present*
PB_ResetUpMEM:BOOL; *//Push Button to Reset Up Memory*
PB_ResetDownMEM:BOOL; *//Push Button to Reset Down Memory*

//Prox Sensors

PRX_PartAtEnd:BOOL; *//Prox Part at Diverter End*
PRX_PartInTransit:BOOL; *//Part Intransit to Diverter*
PRX_TopTable:BOOL; *//Prox Part Present at Up Table*
PRX_BottomTable:BOOL; *//Prox Part Present at Bottom Table*

//Prev Section

Prev_Section_PartReadyToSend:BOOL; *//Previous Section Ready to Send*

//Next Section

Next_Section_PRX_PP_AtStop:BOOL; *//Next Section part Present at Stop*
Next_Section_MTR_On:BOOL; *//Next Section Motor On*

Chapter 4 – TwinCAT 4026, Programming Diverter 01.

```
Next_Section_RequestPart:BOOL;           //Next Section Station Request Part  
END_VAR
```

Diverter Output Variables.

```
VAR_OUTPUT  
  
UpMTR_On:BOOL;           //Up Motor On  
DownMTR_On:BOOL;        //Down Motor On  
MTR_On:BOOL;            //Motor On  
PartAtDiverterEndMem:BOOL; //Part Present at End of Diverter Memory  
PartInTransitOut:BOOL;   //Part In Transit Out Memory  
PartIntransitInMem:BOOL; //Part Intransit In  
PartInTransitUp:BOOL;    //Part In Transit Up Memory  
PartInTransitDown:BOOL;  //Part In Transit Down Memory  
ReadyToReceivePart:BOOL; //Ready to Receive Part Incoming  
END_VAR
```

Diverter Internal Variables.

```
VAR  
  
//Internal Variables  
  
//Condition to Run Motor  
  
i_MTR_ClearToRun:BOOL;           //Clear to Run Motor  
i_MTR_CondToRun:BOOL;           //Auto to Run Motor  
  
//Condition to Run Up Motor  
  
i_UpMTR_ClearToRun:BOOL;        //Clear to Run Up Motor  
i_UpMTR_CondToRun:BOOL;        //Auto to Run Up Motor  
  
//Condition to Run Down Motor  
  
i_DownMTR_ClearToRun:BOOL;      //Clear to Run Down Motor  
i_DownMTR_CondToRun:BOOL;      //Auto to Run Down Motor
```

//Intransit In

i_PartIntransitIn_ONS_R:R_TRIG; //Part Intransit In Rising Edge

i_PartIntransitIn_ONS_F:F_TRIG; //Part Intransit In Falling Edge

//Up Table

i_PartUpTableMem:BOOL; //Part Present Up Table

i_PartUpTable_ONS_R:R_TRIG; //Part Present Up Rising Edge

i_PartUpTable_ONS_F:F_TRIG; //Part Present Up Falling Edge

//Down Table

i_PartDownTableMem:BOOL; //Part Present Down Table

i_PartDownTable_ONS_R:R_TRIG; //Part Present Down Rising Edge

i_PartDownTable_ONS_F:F_TRIG; //Part Present Down Falling Edge

//End Of Diverter

i_PartArrivesAtEnd_ONS: R_TRIG;

//Part Arrived at Stop Rising Trigger (only one shot, only one scan with true value).

i_PartNotAtEND_TMR:TON;

//Not Part present at Stop (Time On Delay Timer)

i_PartAstEND_TMR_ONS:R_Trig;

//Part Present At Stop Rising Trigger (only one shot, only one scan with true value).

i_PartInTransit_ONS:R_Trig;

//Part In Transit One Shot Rising Trigger (only one shot, only one scan with true value).

//Next Section Part Present at Stop

i_Next_Section_PRX_PP_AtStop_ONS:R_TRIG;

//Next Section Part Present at Stop Rising Trigger (only one shot, only one scan with true value).

END_VAR

Diverter Code.

//Part Intransit Up

```
i_PartUpTable_ONS_R(CLK:=PRX_TopTable, Q =>);
```

```
i_PartUpTable_ONS_F(CLK:=PRX_TopTable, Q =>);
```

```
i_PartUpTableMem:= (i_PartUpTable_ONS_R.Q AND UpMTR_On ) OR (i_PartUpTableMem  
AND NOT (i_PartUpTable_ONS_F.Q));
```

```
PartInTransitUp:= (PartAtDiverterEndMem AND UpMTR_On AND NOT  
i_PartUpTable_ONS_R.CLK)OR ( PartInTransitUp AND NOT i_PartUpTable_ONS_R.Q);
```

//Part Intransit Down

```
i_PartDownTable_ONS_R(CLK:=PRX_BottomTable, Q =>);
```

```
i_PartDownTable_ONS_F(CLK:=PRX_BottomTable, Q =>);
```

```
i_PartDownTableMem:= (i_PartDownTable_ONS_R.Q AND DownMTR_On) OR  
(i_PartDownTableMem AND NOT (i_PartDownTable_ONS_F.Q));
```

```
PartInTransitDown:= (PartAtDiverterEndMem AND DownMTR_On AND NOT  
i_PartDownTable_ONS_R.CLK)OR ( PartInTransitDown AND NOT  
i_PartDownTable_ONS_R.Q);
```

//Part Intransit In

```
i_PartIntransitIn_ONS_R(CLK:=PRX_PartInTransit , Q=> );
```

```
i_PartIntransitIn_ONS_F(CLK:=PRX_PartInTransit , Q=> );
```

```
PartIntransitInMem:= (i_PartIntransitIn_ONS_R.Q AND MTR_On) OR (PartIntransitInMem AND  
NOT i_PartIntransitIn_ONS_F.Q);
```

//Part Intransit Out

```
i_PartInTransit_ONS(CLK:=MTR_On AND PartAtDiverterEndMem);
```

```
PartInTransitOut:= i_PartInTransit_ONS.Q OR (PartInTransitOut AND NOT  
i_Next_Section_PRX_PP_AtStop_ONS.Q);
```

//Part at End

```
i_PartArrivesAtEnd_ONS(CLK:=PRX_PartAtEnd);
```

//Part at Diverter Mem

```
PartAtDiverterEndMem:= (PRX_PartAtEnd AND i_PartIntransitIn_ONS_F.Q) OR  
(PartAtDiverterEndMem AND NOT (i_PartInTransit_ONS.Q OR i_PartUpTable_ONS_R.Q OR  
i_PartDownTable_ONS_R.Q));
```

//Request Part

```
ReadyToReceivePart:= NOT PartInTransitOut AND NOT PartInTransitUp AND NOT  
PartInTransitDown AND Prev_Section_PartReadyToSend;
```

```
//Next Section OneShot
```

```
i_Next_Section_PRX_PP_AtStop_ONS(Clk:=Next_Section_PRX_PP_AtStop);
```

```
//Condition to Run Up Motor
```

```
i_UpMTR_ClearToRun:= (NOT PRX_PartInTransit AND PartAtDiverterEndMem) OR (NOT  
PartInTransitInMem OR NOT PartInTransitOut);
```

```
i_UpMTR_CondToRun:= (NOT i_PartUpTableMem AND PartAtDiverterEndMem) OR  
PartInTransitUp; //Uo table get use first
```

```
UpMTR_On:= ((m_Auto AND i_UpMTR_CondToRun) OR (m_Manual AND PB_UpMTR)) AND  
i_UpMTR_ClearToRun;
```

```
//Condition to Run Down Motor
```

```
i_DownMTR_ClearToRun:= (NOT PRX_PartInTransit AND PartAtDiverterEndMem) OR (NOT  
PartInTransitInMem OR NOT PartInTransitOut);
```

```
i_DownMTR_CondToRun:= (NOT i_PartDownTableMem AND PartAtDiverterEndMem AND  
i_PartUpTableMem) OR PartInTransitDown; //Down table get use second
```

```
DownMTR_On:= ((m_Auto AND i_DownMTR_CondToRun) OR (m_Manual AND  
PB_DownMTR)) AND i_DownMTR_ClearToRun;
```

```
//Condition to Determine if the system is clear to start the conveyor
```

```
i_MTR_ClearToRun:= (NOT PartInTransitUp AND NOT PartInTransitDown );
```

```
i_MTR_CondToRun:= (NOT PartAtDiverterEndMem AND ReadyToReceivePart ) OR  
(PartAtDiverterEndMem AND Next_Section_RequestPart AND Next_Section_MTR_On) OR  
(PartInTransitInMem) OR (PartInTransitOut);
```

```
MTR_On:= ((m_Auto AND i_MTR_CondToRun) OR (m_Manual AND PB_RunMTR)) AND  
i_MTR_ClearToRun;
```

```
//Reset Part Present using HMI
```

```
IF (m_Manual AND PB_ResetMEM) THEN
```

```
    PartInTransitOut:=FALSE;
```

```
    PartAtDiverterEndMem:=FALSE;
```

```
    PartInTransitInMem:=FALSE;
```

```
    PartInTransitUp:=FALSE;
    PartInTransitDown:=FALSE;
END_IF;

//Reset Part Present Top using HMI
IF (m_Manual AND PB_ResetUpMEM) THEN
    i_PartUpTableMem:=FALSE;
END_IF;

//Reset Part Present Down using HMI
IF (m_Manual AND PB_ResetUpMEM) THEN
    i_PartDownTableMem:=FALSE;
END_IF;
```

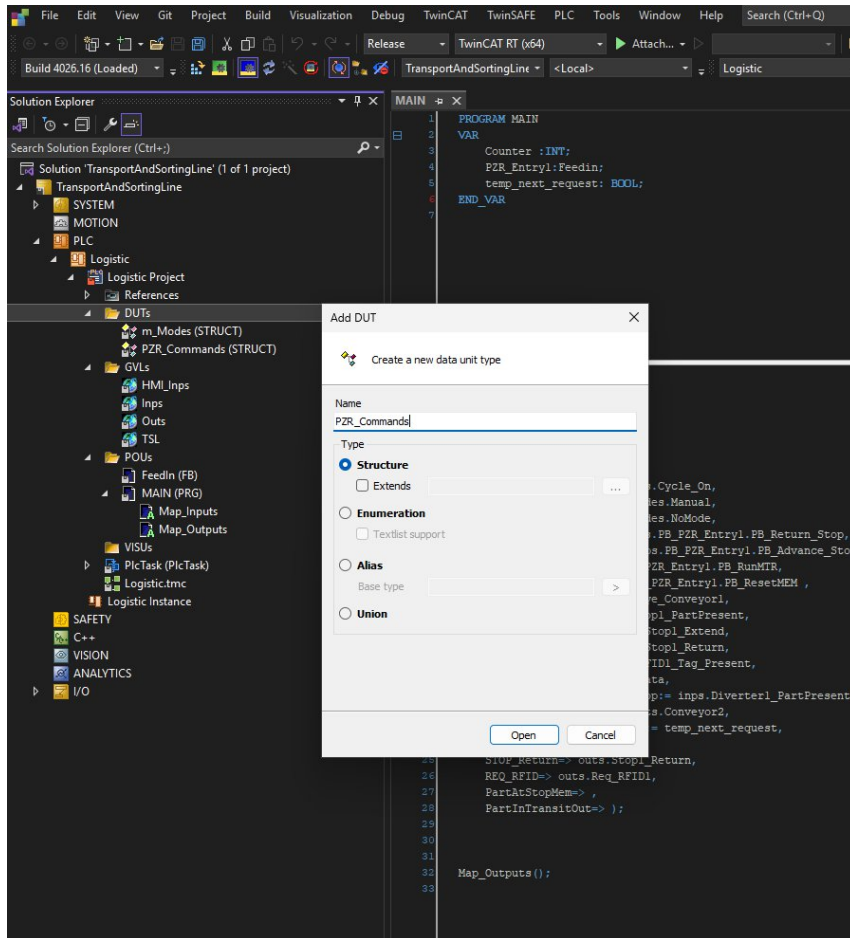

Add a new (DUT) for the Diverter Push Buttons from HMI.

DUT's are user defined data type. You can create your own data types that makes more sense for different functions. We are going to declare one

The DUT is called Diverter_Commands. This data type will have the commands for the Diverter from the HMI Device.

In order to add a DUT.

Right click the folder DUT >> ADD >> DUT.



Diverter_Commands_DUT.

STRUCT

PB_Return_Stop:BOOL; //Push Button to Return Stop

PB_Advance_Stop:BOOL; //Push Button to Advance Stop

PB_RunMTR:BOOL; //Push button to Run Motor

```
PB_ResetMEM:BOOL;           //Push Button to Reset Memory of Part Present
PE_PartIn:BOOL;             //Photo Eye Part at beginning of Conveyor
END_STRUCT
END_TYPE
```

HMI Commands.

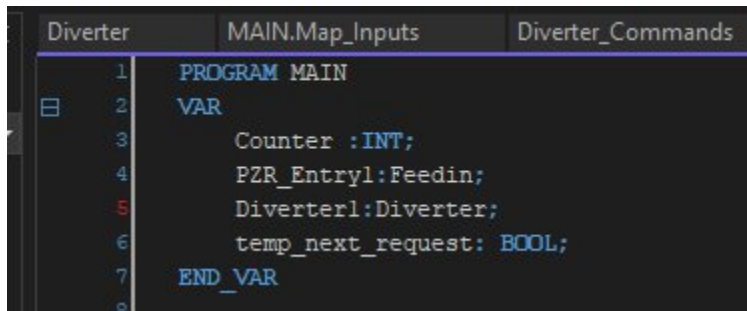
We need to add this new data type to the HMI_Inps Global Variable List.

HMI_Inps GVL.

```
VAR_GLOBAL
    m_Modes:m_Modes;
    PB_PZR_Entry1:PZR_Commands;
    PB_Diverter1:Diverter_Commands;
END_VAR
```

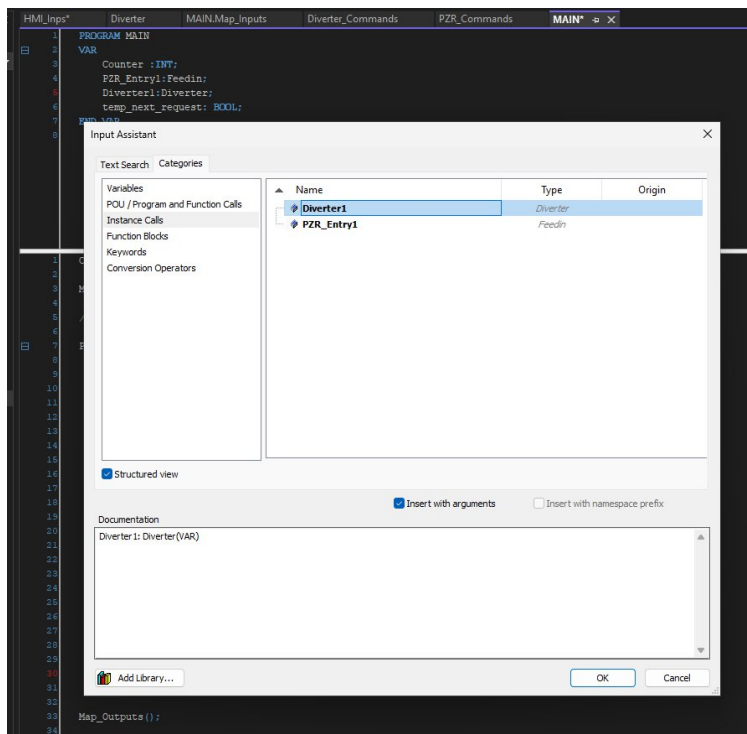
Call Diverter Function Block.

We need to add the Diverter Function Block in the variable section on the Main POU.



```
1 PROGRAM MAIN
2 VAR
3   Counter :INT;
4   PZR_Entry1:Feedin;
5   Diverter1:Diverter;
6   temp_next_request: BOOL;
7 END_VAR
```

There are multiple ways to add Diverter Function Block call on the code. The fastest way is to use the input assistant. Right click on any section on the code and select Input assistant or use F2 key. Select Instance calls and select the Diverter1.



Once selected the Function Block will be called with all the parameters.

```
29
30 Diverter1(
31     m_Auto:= ,
32     m_Manual:= ,
33     m_NoMode:= ,
34     PB_UpMtr:= ,
35     PB_DownMtr:= ,
36     PB_RunMTR:= ,
37     PB_ResetMEM:= ,
38     PB_ResetUpMEM:= ,
39     PB_ResetDownMEM:= ,
40     PRX_PartAtEnd:= ,
41     PRX_PartInTransit:= ,
42     PRX_TopTable:= ,
43     PRX_BottomTable:= ,
44     Prev_Section_PartReadyToSend:= ,
45     Next_Section_PRX_PP_AtStop:= ,
46     Next_Section_MTR_On:= ,
47     Next_Section_RequestPart:= ,
48     UpMTR_On=> ,
49     DownMTR_On=> ,
50     MTR_On=> ,
51     PartAtDiverterEndMem=> ,
52     PartInTransitOut=> ,
53     PartIntransitInMem=> ,
54     PartInTransitUp=> ,
55     PartInTransitDown=> ,
56     ReadyToReceivePart=> );|
57
```

The next step is to assign these parameters.

Assignment of the parameters of the Diverter Function Block call.

//Call of Diverter

```
Diverter1(
m_Auto:= HMI_Inps.m_Modes.Cycle_On,
m_Manual:= HMI_Inps.m_Modes.Manual,
m_NoMode:= HMI_Inps.m_Modes.NoMode,
PB_UpMtr:= HMI_Inps.PB_Diverter1.PB_UpRunMTR,
PB_DownMtr:= HMI_Inps.PB_Diverter1.PB_DownRunMTR,
PB_RunMTR:= HMI_Inps.PB_Diverter1.PB_RunMTR,
PB_ResetMEM:= HMI_Inps.PB_Diverter1.PB_ResetMEM,
```

Chapter 4 – TwinCAT 4026, Programming Diverter 01.

```
PB_ResetUpMEM:= HMI_Inps.PB_Diverter1.PB_UpResetMEM,  
PB_ResetDownMEM:= HMI_Inps.PB_Diverter1.PB_DownResetMEM,  
PRX_PartAtEnd:= inps.Diverter1_PartPresent,  
PRX_PartInTransit:=inps.Diverter1_Part_Intransit ,  
PRX_TopTable:= inps.Storage_Blue_PartPresent,  
PRX_BottomTable:= inps.Storage_White_PartPresent,  
Prev_Section_PartReadyToSend:=PZR_Entry1.PartAtStopMem OR  
PZR_Entry1.PartInTransitOut,  
Next_Section_PRX_PP_AtStop:= inps.Stop2_PartPresent,  
Next_Section_MTR_On:= outs.Conveyor3,  
Next_Section_RequestPart:=temp_next_request ,  
UpMTR_On=>outs.Diverter1_UP ,  
DownMTR_On=> outs.Diverter1_Down,  
MTR_On=> outs.Conveyor2,  
PartAtDiverterEndMem=> ,  
PartInTransitOut=> ,  
PartIntransitInMem=> ,  
PartInTransitUp=> ,  
PartInTransitDown=> ,  
ReadyToReceivePart=> );
```

Previous Logic Change

We need to make small modification on the FeedIn Conveyor Logic. We need to add the Diverter1 variable which indicate to the FeedIn that Diverter1 is ready to receive a part. The variable in red mark this change from the previous logic.

//Call of PZR_Entry1

```
PZR_Entry1(
m_Auto:= HMI_Inps.m_Modes.Cycle_On,
m_Manual:= HMI_Inps.m_Modes.Manual,
m_NoMode:= HMI_Inps.m_Modes.NoMode,
PB_Return_Stop:= HMI_Inps.PB_PZR_Entry1.PB_Return_Stop,
PB_Advance_Stop:= HMI_Inps.PB_PZR_Entry1.PB_Advance_Stop,
PB_RunMTR:= HMI_Inps.PB_PZR_Entry1.PB_RunMTR,
PB_ResetMEM:=HMI_Inps.PB_PZR_Entry1.PB_ResetMEM ,
PE_PartIn:= inps.Photo_Eye_Conveyor1,
PRX_PartAtStop:= inps.Stop1_PartPresent,
PRX_StopAdvanced:= inps.Stop1_Extend,
PRX_StopReturned:= inps.Stop1_Return,
RFID_PartInRage:= inps.RFID1_Tag_Present,
RFID_Data:= inps.RFID1_Data,
Next_Section_PRX_PP_AtStop:= inps.Diverter1_PartPresent,
Next_Section_MTR_On:= outs.Conveyor2,
Next_Section_RequestPart:= Diverter1.ReadyToReceivePart,
MTR_On=>outs.Conveyor1 ,
STOP_Return=> outs.Stop1_Return,
REQ_RFID=> outs.Req_RFID1,
PartAtStopMem=> ,
PartInTransitOut=> );
```

Debug the application.

In this section you will put in practice your knowledge from the previous chapter. Take in consideration that we need to force some variables.

Global Variable List:

HMI_Inps.

We need to force the Cycle_On and Auto.

Outs.

We need to force Conveyor3. This in order to be able to transfer all the parts out of the system.

POU Main.

We need to force the variable **temp_next_request**, in order to indicate to the Diverter1 function block logic that the next section of conveyor is ready to receive the part.